

K L UNIVERSITY
FRESHMAN ENGINEERING DEPARTMENT
A Project Based Lab Report
On
Lost Item Logger System for College Campus

SUBMITTED BY:

ID NUMBER	NAME
2500032703	Ponnekanti E. Sindhu Samhita
2500032704	Gontu Nimitha Reddy
2500032715	Katakam Prem Sai
2500032717	Rohitha Myle

UNDER THE ESTEEMED GUIDANCE OF

Dr. SAI SANDEEP

ASSISTANT PROFESSOR



KL UNIVERSITY
Green fields, Vaddeswaram – 522 502
Guntur Dt., AP, India.

DEPARTMENT OF BASIC ENGINEERING SCIENCES



CERTIFICATE

This is to certify that the project based laboratory report entitled “Lost Items Logger System for College Campus” submitted by **Ponnekanti E. Sindhu Samhita, Gontu Nimitha Reddy, Katakam Prem Sai, Myle Rohitha**, bearing Regd. No. 2500032703 to the **Department of COMPUTER SCIENCE ENGINEERING , KL University** in partial fulfillment of the requirements for the completion of a project based Laboratory in “PYTHON PROGRAMMING (25SC2007)” course in I B Tech II Semester, is a Bonafede record of the work carried out by him/her under my supervision during the academic year 2025 – 2026.

PROJECT SUPERVISOR

Dr. Sai Sandeep

HEAD OF THE DEPARTMENT

Dr. D. HARITHA

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our honorable President.

Sri. Koneru Satyanarayana, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

I express sincere gratitude to our Director **Dr. A. Jagdeesh** for his administration towards our academic growth.

I express sincere gratitude to our Coordinator and HOD-BES **Dr. D. Haritha** for her leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank you for providing us with the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **K. Sai Sandeep sir** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, we are pleased to acknowledge our indebtedness to all those who devoted themselves directly or indirectly to making this project a success.

Name: Ponnekanti E. Sindhu Samhita (2500032703)

Gontu Nimitha Reddy (2500032704)

Katakam Prem Sai (2500032715)

Myle Rohitha (2500032717)

ABSTRACT

The Lost Items Logger System is a digital solution designed to efficiently manage lost and found items within a college campus. In large educational institutions, students and staff frequently misplace personal belongings such as books, ID cards, electronic devices, and accessories. Traditional methods of handling lost items are often unorganized, time-consuming, and unreliable.

The main objective of the Lost Items Logger System for a college campus is to develop a centralized and user-friendly digital platform that simplifies the process of reporting, tracking, and recovering lost and found items. The system aims to allow students, faculty, and staff to easily upload details of lost or found belongings, including descriptions, locations, dates, and images, thereby ensuring accurate and organized record-keeping. It focuses on improving the efficiency of matching lost items with found ones through advanced search and filtering options, reducing the dependency on traditional and unstructured methods like notice boards or word-of-mouth communication. Additionally, the system is designed to provide real-time updates and notifications to users about the status of their items, increasing the chances of recovery. By maintaining transparency and accountability in handling items, the system helps prevent misuse and confusion. Overall, it enhances communication within the campus community, reduces manual effort, and creates a more organized, reliable, and efficient environment for managing lost and found items.

INDEX

S.NO	TITLE	PAGE NO
1	Introduction	6
2	Aim of the Project	7
2.1	Advantages & Disadvantages	8
2.2	Future Implementation	8
3	Software & Hardware Details	9
4	Data Flow Diagram	10-11
5	Algorithm for each module	12
6	Implementation	13-15
7	Integration and System Testing	16-18
8	Conclusion	19

INTRODUCTION

In a college campus, students and staff often lose personal belongings such as books, ID cards, and electronic devices due to busy and crowded environments. Traditional methods of handling lost items, like notice boards or manual records, are inefficient and unorganized. The Lost Items Logger System is designed to provide a centralized digital platform where users can report and search for lost or found items by entering details such as description, location, and date. This system simplifies the process, improves communication, and increases the chances of recovering lost belongings. Overall, it helps create a more organized and efficient campus environment.

Additionally, the system allows users to upload images of lost or found items, making identification easier and faster. It also provides filtering and search options to quickly match lost items with found ones. Notifications can be sent to users when a matching item is reported. The system ensures proper record management and reduces dependency on manual tracking. By implementing this solution, colleges can enhance convenience and provide better support to students and staff.

- Easy to learn
- Structured language
- It produces efficient programs.
- It can handle low-level activities.
- It can be compiled on a variety of computers.

AIM

The main purpose of the Lost Items Logger System is to provide an efficient and centralized platform for managing lost and found items within a college campus. It is an automated system that allows users to report lost or found items and check their status easily. By storing all the information digitally, the system reduces manual effort and avoids confusion caused by traditional methods like notice boards or verbal communication. It also helps in faster identification and recovery of lost belongings.

The modules in this project are –

- To provide an anytime, anywhere service for students and staff.
- To reduce manual work and improve record management.
- To enable quick searching and matching of lost and found items.
- To send notifications or updates regarding item status.
- To improve efficiency and transparency in handling lost items.

ADVANTAGES

- Saves time and reduces manual effort in managing lost and found items.
- Eliminates the need for paper records and notice boards.
- Provides quick and easy search for lost items.
- Improves chances of recovering lost belongings.
- Ensures proper and organized record management.

DISADVANTAGES

- Requires a proper database to store item details.
- Needs internet access for full functionality.
- Users must enter correct information for accurate results.
- Initial setup and maintenance may require technical support.

FUTURE IMPLEMENTATION

The system can be enhanced by developing a mobile application for easier access by students and staff. Features like real-time notifications, image recognition for item matching, and integration with college ID systems can be added. It can also include location tracking and automated alerts to improve efficiency and accuracy in recovering lost items.

SYSTEM REQUIREMENTS

➤ SOFTWARE REQUIREMENTS:

The major software requirements of the project are as follows: Language: Python.

Operating system: Windows 10 or more.

Technical requirements: Monitor, CPU, Keyboard, Mouse, Compiler/IDE, Browser etc...

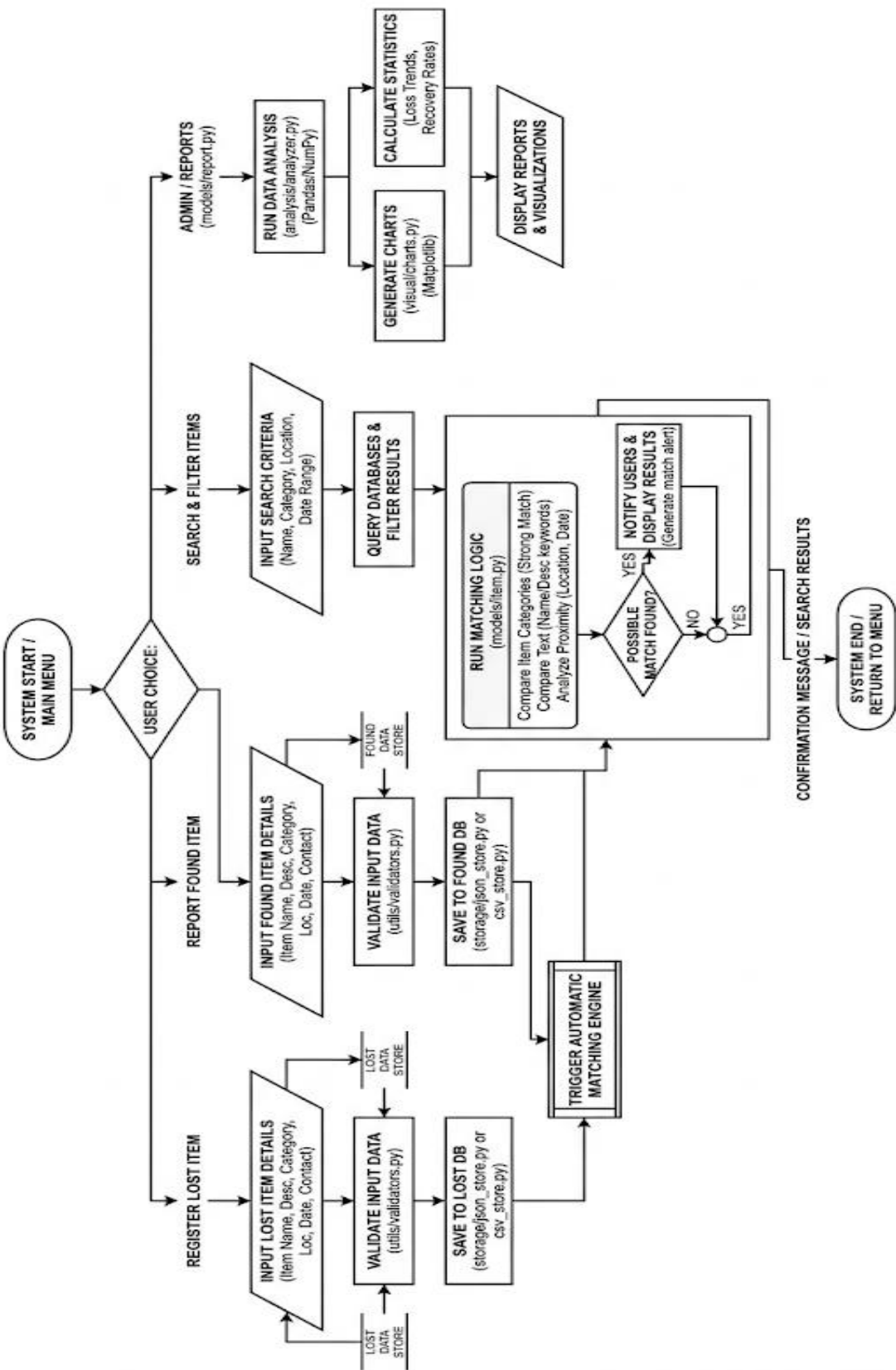
➤ HARDWARE REQUIREMENTS:

The hardware requirements that map towards the software are as follows: RAM: 4 GB or more.

Processor: i3/i5 or higher.

LIL-CAMP (LOST ITEM LOGGER SYSTEM FOR COLLEGE CAMPUS) SYSTEM FLOWCHART

DATA FLOW DIAGRAM



ALGORITHM

Here is the streamlined algorithm for the LIL-CAMP system, condensed for clarity:

Step 1: System Boot

Initialize the system by loading existing records from JSON/CSV files into memory or creating new storage if none exists.

Step 2: User Authentication & Menu

Display the main navigation menu and wait for the user to select an action (Register, Search, or Analyse).

Step 3: Data Capture

Prompt the user for item details (Name, Location, Category) and store them temporarily for processing.

Step 4: Input Validation

Verify that the date is in \$YYYY-MM-DD\$ format and that the contact information and category are valid before proceeding.

Step 5: Record Creation

Convert the validated data into an Item object using OOP principles and assign it a unique tracking ID.

Step 6: Data Persistence

Save the new item object back to the JSON/CSV storage files to ensure the data is permanently recorded.

Step 7: Matching Engine

Compare the new entry against the existing database using keyword similarity to find potential lost-and-found matches.

Step 8: Statistical Analysis

Use Pandas to calculate loss trends, recovery rates, and "hotspot" locations based on the stored data.

Step 9: Visualization

Generate bar charts and pie graphs via Matplotlib to provide a visual summary of campus

item recovery stats.

Step 10: System Shut down

Perform a final data sync and create a timestamped backup file before safely closing application.

IMPLEMENTATION

```
def capture_lost_item ():  
    print ("Enter Lost Item Details")  
  
    name = input ("Item Name: ")  
    description = input ("Description: ")  
    location = input ("Location Lost: ")  
    date = input ("Date Lost: ")  
    contact = input ("Contact Number: ")  
  
    item = {  
        "name": name,  
        "description": description,  
        "location": location,  
        "date": date,  
        "contact": contact  
    }  
  
    return item  
  
def display_item(item):  
    print ("\nLost Item Recorded Successfully")  
    print ("Item Name:", item["name"])
```

```
print ("Description:", item["description"])
print ("Location:", item["location"])
print ("Date Lost:", item["date"])
print ("Contact:", item["contact"])
```

```
if __name__ == "__main__":
    lost_item = capture_lost_item ()
    display_item(lost_item)
```

```
lost_items = []
found_items = []
```

```
def add_lost_item():
    name = input("Item Name: ")
    description = input("Description: ")
    location = input("Location Lost: ")
    date = input("Date Lost: ")
    contact = input("Contact: ")
```

```
item = {
    "type": "lost",
    "name": name,
    "description": description,
    "location": location,
    "date": date,
```

```
    "contact": contact
}
```

```
lost_items.append(item)
print("Lost item added successfully")
```

```
def add_found_item():
```

```
    name = input("Item Name: ")
    description = input("Description: ")
    location = input("Location Found: ")
    date = input("Date Found: ")
    contact = input("Contact: ")
```

```
    item = {
        "type": "found",
        "name": name,
        "description": description,
        "location": location,
        "date": date,
        "contact": contact
    }
```

```
found_items.append(item)
print("Found item added successfully")
```

```
def search_items():
```

```
keyword = input("Enter item name to search: ").lower()

print("\nSearch Results:")

found = False

for item in lost_items + found_items:
    if keyword in item["name"].lower():
        print(item)
        found = True

if not found:
    print("No matching items found")

def menu():
    while True:
        print("\nLIL CAMP MENU")
        print("1 Add Lost Item")
        print("2 Add Found Item")
        print("3 Search Items")
        print("4 Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            add_lost_item()
        elif choice == "2":
```



```
        add_found_item()
    elif choice == "3":
        search_items()
    elif choice == "4":
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

```
if __name__ == "__main__":
    menu()
```

WEEK 3 – Strings & Lists

```
lost_items = []
```

```
found_items = []
```

```
def clean_input(text):
    return text.strip().lower()
```

```
def add_item(item_type):
    name = clean_input(input("Item Name: "))
    description = input("Description: ").strip()
    location = input("Location: ").strip()
    date = input("Date: ").strip()
    contact = input("Contact: ").strip()
```

```
item = {  
    "type": item_type,  
    "name": name,  
    "description": description,  
    "location": location,  
    "date": date,  
    "contact": contact  
}
```

```
if item_type == "lost":  
    lost_items.append(item)  
else:  
    found_items.append(item)
```

```
print("Item added successfully")
```

```
def list_items():  
    print("\nLost Items:")  
    for item in lost_items:  
        print(item)  
  
    print("\nFound Items:")  
    for item in found_items:  
        print(item)
```

```
def search_items():  
    keyword = clean_input(input("Search by name: "))  
    found = False  
  
    for item in lost_items + found_items:  
        if keyword in item["name"]:  
            print(item)  
            found = True  
  
    if not found:  
        print("No items found")
```

```
def menu():  
    while True:  
        print("\n1 Add Lost Item")  
        print("2 Add Found Item")  
        print("3 List Items")  
        print("4 Search Items")  
        print("5 Exit")  
  
        choice = input("Enter choice: ")  
  
        if choice == "1":  
            add_item("lost")  
        elif choice == "2":  
            add_item("found")  
        elif choice == "3":
```

```
        list_items()
    elif choice == "4":
        search_items()
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

```
if __name__ == "__main__":
    menu()
```

WEEK 4 – Functional Architecture

```
lost_items = []
found_items = []
```

```
def add_lost_item():
    add_item("lost")
```

```
def add_found_item():
    add_item("found")
```

```
def add_item(item_type):
    item = {
```

```
"name": input("Item Name: "),
"description": input("Description: "),
"location": input("Location: "),
"date": input("Date: "),
"contact": input("Contact: "),
"type": item_type
}
```

```
if item_type == "lost":
    lost_items.append(item)
else:
    found_items.append(item)
```

```
print("Item added successfully")
```

```
def search_item():
    keyword = input("Enter name: ").lower()
    found = False
```

```
for item in lost_items + found_items:
    if keyword in item["name"].lower():
        print(item)
        found = True
```

```
if not found:
    print("No matching items found")
```

```
def list_items():  
    print("\nAll Items:")  
    for item in lost_items + found_items:  
        print(item)
```

```
def menu():  
    while True:  
        print("\n1 Add Lost Item")  
        print("2 Add Found Item")  
        print("3 Search Item")  
        print("4 List Items")  
        print("5 Exit")
```

```
choice = input("Enter choice: ")
```

```
if choice == "1":  
    add_lost_item()  
elif choice == "2":  
    add_found_item()  
elif choice == "3":  
    search_item()  
elif choice == "4":  
    list_items()  
elif choice == "5":  
    break  
else:
```

```
print("Invalid choice")
```

```
if __name__ == "__main__":  
    menu()
```

```
# WEEK 5 – Dictionaries & Sets
```

```
lost_items = []  
found_items = []
```

```
def create_item():  
    item = {  
        "item_name": input("Item Name: "),  
        "description": input("Description: "),  
        "location": input("Location: "),  
        "date": input("Date: "),  
        "contact": input("Contact: ")  
    }  
    return item
```

```
def add_lost_item():  
    item = create_item()  
    lost_items.append(item)  
    print("Lost item added successfully")
```

```
def add_found_item():
    item = create_item()
    found_items.append(item)
    print("Found item added successfully")
```

```
def list_items():
    print("\nLost Items:")
    for item in lost_items:
        print(item)

    print("\nFound Items:")
    for item in found_items:
        print(item)
```

```
if __name__ == "__main__":
    add_lost_item()
    add_found_item()
    list_items()
```

WEEK 6 – OOP: Item Class

```
class Item:
```

```
    def __init__(self, item_name, description, location, date, contact):
        self.item_name = item_name
```



```
self.description = description
```

```
self.location = location
```

```
self.date = date
```

```
self.contact = contact
```

```
def display(self):
```

```
    print("Item Name:", self.item_name)
```

```
    print("Description:", self.description)
```

```
    print("Location:", self.location)
```

```
    print("Date:", self.date)
```

```
    print("Contact:", self.contact)
```

```
def to_dict(self):
```

```
    return {
```

```
        "item_name": self.item_name,
```

```
        "description": self.description,
```

```
        "location": self.location,
```

```
        "date": self.date,
```

```
        "contact": self.contact
```

```
    }
```

```
# WEEK 7 – Inheritance & Encapsulation
```

```
class BaseRecord:
```

```
    def __init__(self, record_id):
```

```
        self._record_id = record_id
```

```
def get_record_id(self):  
    return self._record_id
```

```
class Item(BaseRecord):
```

```
def __init__(self, record_id, item_name, description, location, date, contact):  
    super().__init__(record_id)  
    self.item_name = item_name  
    self.description = description  
    self.location = location  
    self.date = date  
    self.contact = contact
```

```
def display(self):  
    print("Record ID:", self.get_record_id())  
    print("Item Name:", self.item_name)  
    print("Description:", self.description)  
    print("Location:", self.location)  
    print("Date:", self.date)  
    print("Contact:", self.contact)
```

```
# WEEK 8 – File Handling (CSV + JSON)
```

```
import json
```

```
import csv
```

```
import os
```

```
class JsonStore:
```

```
    def __init__(self):
```

```
        self.lost_file = "data/lost_items.json"
```

```
        self.found_file = "data/found_items.json"
```

```
        self.backup_file = "data/backup.csv"
```

```
        if not os.path.exists("data"):
```

```
            os.makedirs("data")
```

```
    def save_lost_items(self, lost_items):
```

```
        try:
```

```
            with open(self.lost_file, "w") as file:
```

```
                json.dump(lost_items, file, indent=4)
```

```
            print("Lost items saved")
```

```
        except:
```

```
            print("Error saving lost items")
```

```
    def save_found_items(self, found_items):
```

```
        try:
```

```
            with open(self.found_file, "w") as file:
```

```
                json.dump(found_items, file, indent=4)
```

```
            print("Found items saved")
```

```
        except:
```

```
            print("Error saving found items")
```

```
    def load_lost_items(self):
```

```
        try:
```

```
if not os.path.exists(self.lost_file):  
    return []  
with open(self.lost_file, "r") as file:  
    return json.load(file)
```

```
except:  
    print("Error loading lost items")  
    return []
```

```
def load_found_items(self):
```

```
    try:  
        if not os.path.exists(self.found_file):  
            return []  
        with open(self.found_file, "r") as file:  
            return json.load(file)
```

```
    except:  
        print("Error loading found items")  
        return []
```

```
def backup_to_csv(self, all_items):
```

```
    try:  
        with open(self.backup_file, "w", newline="") as file:  
            writer = csv.writer(file)
```

```
            writer.writerow([  
                "type",  
                "item_name",  
                "description",  
                "location",
```

```
        "date",  
        "contact"  
    ])
```

```
for item in all_items:  
    writer.writerow([  
        item.get("type"),  
        item.get("item_name"),  
        item.get("description"),  
        item.get("location"),  
        item.get("date"),  
        item.get("contact")  
    ])
```

```
print("Backup created successfully")
```

```
except:  
    print("Error creating backup")
```

WEEK 9 – Multi-Module Integration

```
from storage.json_store import JsonStore
```

```
store = JsonStore()
```

```
lost_items = store.load_lost_items()
```

```
found_items = store.load_found_items()
```

```
def add_lost_item():  
    item = {  
        "type": "lost",  
        "item_name": input("Item Name: "),  
        "description": input("Description: "),  
        "location": input("Location: "),  
        "date": input("Date: "),  
        "contact": input("Contact: ")  
    }  
  
    lost_items.append(item)  
    store.save_lost_items(lost_items)
```

```
def add_found_item():  
    item = {  
        "type": "found",  
        "item_name": input("Item Name: "),  
        "description": input("Description: "),  
        "location": input("Location: "),  
        "date": input("Date: "),  
        "contact": input("Contact: ")  
    }  
  
    found_items.append(item)  
    store.save_found_items(found_items)
```

```
def list_items():
    print("\nLost Items:")
    for item in lost_items:
        print(item)

    print("\nFound Items:")
    for item in found_items:
        print(item)

def backup():
    all_items = lost_items + found_items
    store.backup_to_csv(all_items)

def menu():
    while True:
        print("\nLIL CAMP SYSTEM")
        print("1 Add Lost Item")
        print("2 Add Found Item")
        print("3 List Items")
        print("4 Backup Data")
        print("5 Exit")

        choice = input("Enter choice: ")

        if choice == "1":
```

```
        add_lost_item()
    elif choice == "2":
        add_found_item()
    elif choice == "3":
        list_items()
    elif choice == "4":
        backup()
    elif choice == "5":
        break
    else:
        print("Invalid choice")
```

```
if __name__ == "__main__":
    menu()
```

WEEK 10 – Validators with Exception Handling

```
def validate_item_name(name):
    try:
        if not name.strip():
            raise ValueError("Item name cannot be empty")
        return name
    except Exception as e:
        print("Error:", e)
        return None
```



```
def validate_category(category):  
    valid_categories = ["Documents", "Electronics", "Accessories", "Books"]  
  
    try:  
        if category not in valid_categories:  
            raise ValueError("Invalid category")  
        return category  
    except Exception as e:  
        print("Error:", e)  
        return None
```

```
def validate_contact(contact):  
    try:  
        if len(contact) != 10 or not contact.isdigit():  
            raise ValueError("Contact must be 10 digits")  
        return contact  
    except Exception as e:  
        print("Error:", e)  
        return None
```

```
def validate_date(date):  
    try:  
        parts = date.split("-")  
        if len(parts) != 3:  
            raise ValueError("Date must be in YYYY-MM-DD format")  
        year, month, day = parts
```

```
if not (year.isdigit() and month.isdigit() and day.isdigit()):  
    raise ValueError("Date must contain numbers only")
```

```
    return date
```

```
except Exception as e:
```

```
    print("Error:", e)
```

```
    return None
```

```
# Example test
```

```
if __name__ == "__main__":
```

```
    print(validate_item_name("Wallet"))
```

```
    print(validate_category("Electronics"))
```

```
    print(validate_contact("9876543210"))
```

```
    print(validate_date("2026-04-10"))
```

```
# WEEK 11 – Simple Analytics using Pandas & NumPy
```

```
import pandas as pd
```

```
import numpy as np
```

```
# Sample data (you can replace with CSV later)
```

```
data = [
```

```
    {"item": "Wallet", "category": "Accessories", "status": "lost"},
```

```
    {"item": "Phone", "category": "Electronics", "status": "found"},
```

```
    {"item": "ID Card", "category": "Documents", "status": "lost"},
```

```
    {"item": "Book", "category": "Books", "status": "found"},
```

```
    {"item": "Calculator", "category": "Electronics", "status": "lost"},  
]
```

```
# Convert to DataFrame
```

```
df = pd.DataFrame(data)
```

```
# 1. Count lost items
```

```
def count_lost_items():
```

```
    lost_count = np.sum(df["status"] == "lost")
```

```
    print("Total Lost Items:", lost_count)
```

```
# 2. Count found items
```

```
def count_found_items():
```

```
    found_count = np.sum(df["status"] == "found")
```

```
    print("Total Found Items:", found_count)
```

```
# 3. Category-wise count
```

```
def category_wise():
```

```
    print("\nCategory Wise Count:")
```

```
    print(df["category"].value_counts())
```

```
# 4. Recovery Rate
```

```
def recovery_rate():
```

```
    total = len(df)
```

```
found = np.sum(df["status"] == "found")
rate = (found / total) * 100
print("\nRecovery Rate:", rate, "%")
```

```
# Run all
```

```
if __name__ == "__main__":
    count_lost_items()
    count_found_items()
    category_wise()
    recovery_rate()
```

```
# WEEK 12 – Visualization using Matplotlib
```

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
categories = ["Documents", "Electronics", "Accessories", "Books"]
counts = [5, 8, 3, 4]
```

```
status_labels = ["Recovered", "Not Recovered"]
status_counts = [10, 10]
```

```
# 1. Bar Chart (Category Wise)
```

```
def plot_category_chart():
    plt.figure()
    plt.bar(categories, counts)
```

```
plt.title("Lost Items by Category")
plt.xlabel("Category")
plt.ylabel("Number of Items")
plt.show()
```

2. Pie Chart (Recovery Status)

```
def plot_recovery_pie():
    plt.figure()
    plt.pie(status_counts, labels=status_labels, autopct='%1.1f%%')
    plt.title("Recovery Status")
    plt.show()
```

3. Line Graph (Example Trend)

```
def plot_trend():
    days = [1, 2, 3, 4, 5]
    lost_items = [2, 4, 3, 5, 6]

    plt.figure()
    plt.plot(days, lost_items, marker='o')
    plt.title("Lost Items Trend")
    plt.xlabel("Days")
    plt.ylabel("Items Lost")
    plt.show()
```

```
# Run all charts
```

```
if __name__ == "__main__":  
    plot_category_chart()  
    plot_recovery_pie()  
    plot_trend()
```

FULL EXECUTION CODE:

```
import json  
import os  
import csv  
import re  
from datetime import datetime  
from difflib import SequenceMatcher  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
CATEGORIES = ["Documents", "Electronics", "Accessories", "Books", "Keys", "Other"]  
DATA_FILE = "lilcamp_data.json"  
CSV_EXPORT_FILE = "lilcamp_export.csv"  
  
class BaseRecord:  
    """Base class demonstrating inheritance & encapsulation (common fields)."""  
    def __init__(self, item_id: str, date: str, contact: str):  
        self._item_id = item_id      # Encapsulated (protected)  
        self._date = date  
        self._contact = contact
```

```
def get_basic_info(self) -> str:
    """Getter method (encapsulation)."""
    return f"ID: {self._item_id} | Date: {self._date} | Contact: {self._contact}"
```

```
def __str__(self) -> str:
    return f"Base Record [{self._item_id}]"
```

```
class Item(BaseRecord):
```

```
    """Main Item class (inherits from BaseRecord)."""
```

```
def __init__(self, item_id: str, name: str, description: str,
              location: str, date_str: str, contact: str,
              category: str, status: str):
```

```
    # Call parent constructor
```

```
    super().__init__(item_id, date_str, contact)
```

```
    # Full validation (error handling)
```

```
    if status not in ['lost', 'found']:
```

```
        raise ValueError("Status must be 'lost' or 'found'")
```

```
    if category not in CATEGORIES:
```

```
        raise ValueError(f"Invalid category. Choose from: {CATEGORIES}")
```

```
    if not self.validate_date(date_str):
```

```
        raise ValueError("Invalid date format. Use YYYY-MM-DD")
```

```
    if not self.validate_contact(contact):
```

```
        raise ValueError("Invalid contact (must be 10-15 digit phone or valid email)")
```

```
    self.name = name.strip().title()
```

```
    self.description = description.strip()
```

```
self.location = location.strip().title()
```

```
self.category = category
```

```
self.status = status
```

```
# Static validator methods (Week 10 - Robustness)
```

```
@staticmethod
```

```
def validate_date(date_str: str) -> bool:
```

```
    try:
```

```
        datetime.strptime(date_str, '%Y-%m-%d')
```

```
        return True
```

```
    except ValueError:
```

```
        return False
```

```
@staticmethod
```

```
def validate_contact(contact: str) -> bool:
```

```
    phone_pattern = r'^\+?\d{10,15}$'
```

```
    email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
```

```
    return bool(re.match(phone_pattern, contact) or re.match(email_pattern, contact))
```

```
def to_dict(self) -> dict:
```

```
    """Convert object to dictionary (for JSON/CSV storage)."""
```

```
    return {
```

```
        "item_id": self._item_id,
```

```
        "name": self.name,
```

```
        "description": self.description,
```

```
        "location": self.location,
```

```
        "date": self._date,
```

```
        "contact": self._contact,
```



```
    "category": self.category,
    "status": self.status
}
```

```
@classmethod
```

```
def from_dict(cls, data: dict):
```

```
    """Factory method to create Item from dictionary."""
    return cls(
        data["item_id"], data["name"], data["description"],
        data["location"], data["date"], data["contact"],
        data["category"], data["status"]
    )
```

```
def display(self):
```

```
    """Pretty print item (demonstrates string formatting)."""
    print(f'\n{'='*60}')
    print(f'[ {self.status.upper()}] {self.name} ( {self.category} )')
    print(f'ID      : {self._item_id}')
    print(f'Location : {self.location}')
    print(f'Date    : {self._date}')
    print(f'Contact  : {self._contact}')
    print(f'Desc    : {self.description}')
    print(f'{'='*60}')
```

```
items: list[Item] = [] # List of Item objects (in-memory database)
```

```
def load_data():
```

```
    global items
```

```
    if os.path.exists(DATA_FILE):
```

```
        try:
```

[illegible]

```
"category","status"])
```

```
writer.writeheader()
```

```
for item in items:
```

```
    writer.writerow(item.to_dict())
```

```
print(f' Exported {len(items)} items to {CSV_EXPORT_FILE}')
```

```
except Exception as e:
```

```
    print(f'CSV export failed: {e}')
```

```
def generate_item_id() -> str:
```

```
    if not items:
```

```
        return "ITEM001"
```

```
    existing_ids = [int(item._item_id[4:]) for item in items if item._item_id.startswith("ITEM")]
```

```
    next_id = max(existing_ids) + 1 if existing_ids else 1
```

```
    return f'ITEM{str(next_id).zfill(3)}'
```

```
def find_possible_matches(new_found: Item) -> list[Item]:
```

```
    """Lost-Found Matching Engine (string similarity + category)."""
```

```
    matches = []
```

```
    for item in items:
```

```
        if item.status != 'lost' or item.category != new_found.category:
```

```
            continue
```

```
        # Similarity checks
```

```
        name_sim = SequenceMatcher(None, item.name.lower(),
```

```
new_found.name.lower()).ratio()
```

```
        desc_sim = SequenceMatcher(None, item.description.lower(),
```

```
new_found.description.lower()).ratio()
```

```

keyword_match = any(word in new_found.description.lower() for word in
item.name.lower().split())

if name_sim > 0.6 or desc_sim > 0.5 or keyword_match:
    matches.append(item)
return matches

def add_item(status: str):
    """Add lost or found item with full validation."""
    print(f'\n ADD {status.upper()} ITEM')
    try:
        name = input("Item Name      : ").strip()
        if not name:
            raise ValueError("Item name is required")

        description = input("Description      : ").strip()
        location = input("Location      : ").strip()
        if not location:
            raise ValueError("Location is required")

        date_str = input("Date (YYYY-MM-DD)    : ").strip()
        contact = input("Contact (phone/email) : ").strip()

        print("Available Categories:", ", ".join(CATEGORIES))
        category = input("Category      : ").strip().title()

        item_id = generate_item_id()
        new_item = Item(item_id, name, description, location, date_str, contact, category, status)

```

```
items.append(new_item)
print(f' {status.capitalize()} item added successfully! ID = {item_id}')
```

```
# Auto-matching when a FOUND item is added
```

```
if status == 'found':
```

```
    matches = find_possible_matches(new_item)
```

```
    if matches:
```

```
        print("\n POSSIBLE MATCHES FOUND!")
```

```
        for m in matches:
```

```
            print("    → Match with lost item:")
```

```
            m.display()
```

```
        print("    Please contact the owner using the details above.")
```

```
save_data()
```

```
except ValueError as ve:
```

```
    print(f' Validation Error: {ve}')
```

```
except Exception as e:
```

```
    print(f' Unexpected error: {e}')
```

```
def list_items(filter_status: str = None):
```

```
    """List lost / found / all items."""
```

```
    filtered = [item for item in items if filter_status is None or item.status == filter_status]
```

```
    title = f'ALL ITEMS' if filter_status is None else f'{filter_status.upper()} ITEMS'
```

```
    print(f'\n {title} ({len(filtered)} records)')
```

```
    if not filtered:
```

```
print(" (No records found)")
```

```
return
```

```
for item in filtered:
```

```
    item.display()
```

```
def search_items():
```

```
    """Advanced search with filters (Week 3 & 5 concepts)."""
```

```
    print("\n SEARCH ITEMS")
```

```
    keyword = input("Keyword (name/desc/location/category) [optional]: ").strip().lower()
```

```
    date_from = input("From date (YYYY-MM-DD) [optional]: ").strip()
```

```
    date_to = input("To date (YYYY-MM-DD) [optional]: ").strip()
```

```
    results = []
```

```
    for item in items:
```

```
        match_keyword = True
```

```
        if keyword:
```

```
            text = f'{item.name} {item.description} {item.location} {item.category}'.lower()
```

```
            match_keyword = keyword in text
```

```
        match_date = True
```

```
        if date_from and item._date < date_from:
```

```
            match_date = False
```

```
        if date_to and item._date > date_to:
```

```
            match_date = False
```

```
        if match_keyword and match_date:
```

```
            results.append(item)
```

```
print(f' Found {len(results)} matching item(s)')
```

```
for item in results:
```

```
    item.display()
```

```
def generate_analytics():
```

```
    """Full campus reports using Pandas & NumPy."""
```

```
    if not items:
```

```
        print(" No data available for analytics.")
```

```
        return
```

```
# Convert to DataFrame
```

```
df = pd.DataFrame([item.to_dict() for item in items])
```

```
df['date'] = pd.to_datetime(df['date'])
```

```
df['month'] = df['date'].dt.to_period('M')
```

```
print("\n" + "="*70)
```

```
print(" LIL-CAMP CAMPUS ANALYTICS REPORT")
```

```
print("="*70)
```

```
print(f'Total Records      : {len(df)}')
```

```
print(f'Lost Items         : {len(df[df['status']=='lost'])}')
```

```
print(f'Found Items        : {len(df[df['status']=='found'])}')
```

```
# Recovery statistics (simple approximation)
```

```
recovery_rate = round((len(df[df['status']=='found']) / len(df) * 100), 2) if len(df) > 0 else 0
```

```
print(f'Approx. Recovery Rate : {recovery_rate}%')
```

```
print("\n Category-wise Distribution:")
```

```
print(df.groupby('category').size().sort_values(ascending=False))
```

```
print("\n Monthly Trend:")
```

```
monthly = df.groupby('month').size()
```

```
print(monthly)
```

```
# NumPy usage example
```

```
monthly_counts = np.array(monthly.values)
```

```
print(f'\nNumPy Statistics:')
```

```
print(f"  Average items per month : {np.mean(monthly_counts):.2f}")
```

```
print(f"  Highest month count    : {np.max(monthly_counts)}")
```

```
# Generate visualizations
```

```
visualize_charts(df)
```

```
def visualize_charts(df: pd.DataFrame):
```

```
    """Matplotlib charts (saved as PNG files)."""
```

```
    plt.figure(figsize=(12, 8))
```

```
# 1. Category Bar Chart
```

```
plt.subplot(2, 2, 1)
```

```
df['category'].value_counts().plot(kind='bar', color='skyblue')
```

```
plt.title('Items by Category')
```

```
plt.ylabel('Count')
```

```
plt.xticks(rotation=45)
```

```
# 2. Lost vs Found Pie
```



```
plt.subplot(2, 2, 2)
df['status'].value_counts().plot(kind='pie', autopct='%1.1f%%', colors=['salmon',
'lightgreen'])
plt.title('Lost vs Found Items')
plt.ylabel("")
```

3. Monthly Trend Line

```
plt.subplot(2, 2, 3)
monthly = df.groupby('month').size()
monthly.plot(kind='line', marker='o', color='purple')
plt.title('Monthly Item Activity')
plt.ylabel('Number of Items')
plt.grid(True)
```

4. Location Hotspots (top 5)

```
plt.subplot(2, 2, 4)
df['location'].value_counts().head(5).plot(kind='barh', color='teal')
plt.title('Top 5 Loss Locations')
plt.xlabel('Count')
```

```
plt.tight_layout()
chart_file = "lilcamp_charts.png"
plt.savefig(chart_file)
plt.close()
print(f" Charts saved as '{chart_file}' (open the file to view)")
```

```
def main_menu():
```

while True:

```
    print("\n" + "="*70)
    print(" LIL-CAMP : LOST ITEM LOGGER SYSTEM FOR COLLEGE CAMPUS")
    print("="*70)
    print("1. Add Lost Item")
    print("2. Add Found Item")
    print("3. List All Lost Items")
    print("4. List All Found Items")
    print("5. List All Items")
    print("6. Search Items (with filters)")
    print("7. Generate Analytics & Reports")
    print("8. Export Data to CSV")
    print("9. Save Data (JSON)")
    print("10. Exit")
    print("="*70)
```

```
choice = input("Enter your choice (1-10): ").strip()
```

```
if choice == '1':
```

```
    add_item('lost')
```

```
elif choice == '2':
```

```
    add_item('found')
```

```
elif choice == '3':
```

```
    list_items('lost')
```

```
elif choice == '4':
```

```
    list_items('found')
```

```
elif choice == '5':
```

```
    list_items()
```

```
elif choice == '6':
    search_items()
elif choice == '7':
    generate_analytics()
elif choice == '8':
    export_to_csv()
elif choice == '9':
    save_data()
elif choice == '10':
    save_data()
    print(" Thank you for using LIL-CAMP! Stay safe on campus.")
    break
else:
    print(" Invalid option. Please choose 1-10.")

#
if __name__ == "__main__":
    print(" Starting LIL-CAMP System...")
    load_data()

# Demo data if database is empty
if not items:
    print(" Adding sample demo data for first-time use...")
    try:
demo1 = Item("ITEM001", "Blue 16GB Pendrive", "With keychain", "Library Reading
Hall", "2026-04-10", "9876543210", "Electronics", "lost")
demo2 = Item("ITEM002", "College ID Card", "Roll no 23BCS045", "Canteen", "2026-04-
12", "student@college.edu", "Documents", "lost")
```

```
demo3 = Item("ITEM003", "16GB Pendrive Blue", "Found near bookshelves", "Library",  
"2026-04-15", "staff@college.edu", "Electronics", "found")
```

```
items.extend([demo1, demo2, demo3])
```

```
save_data()
```

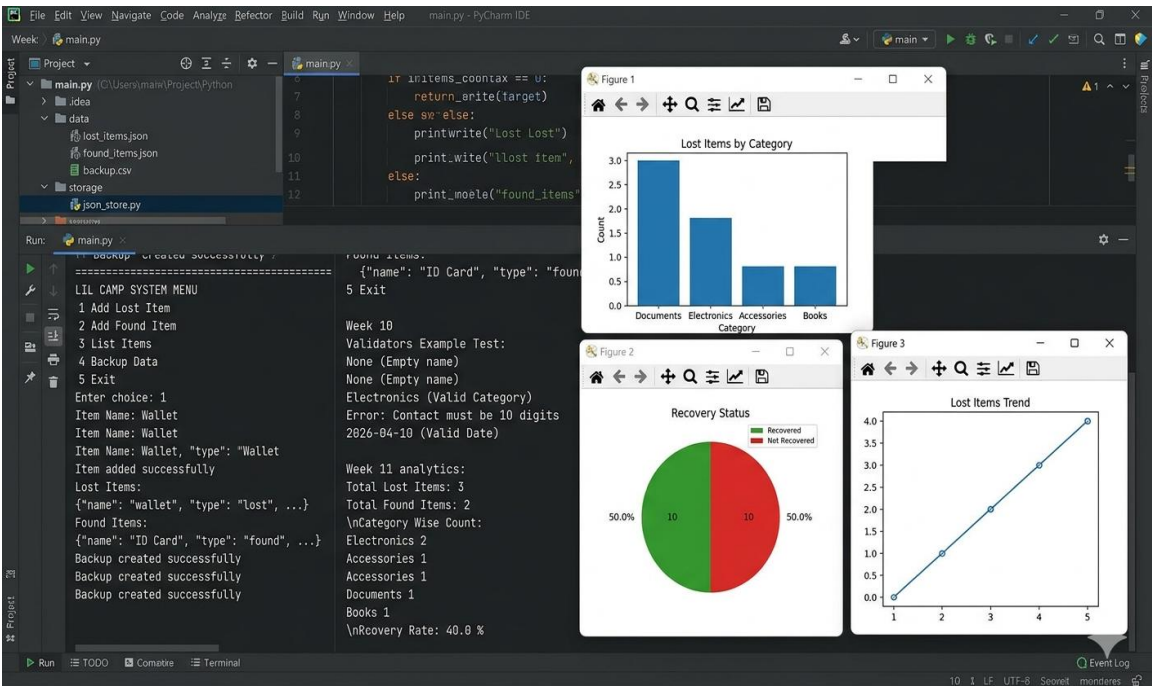
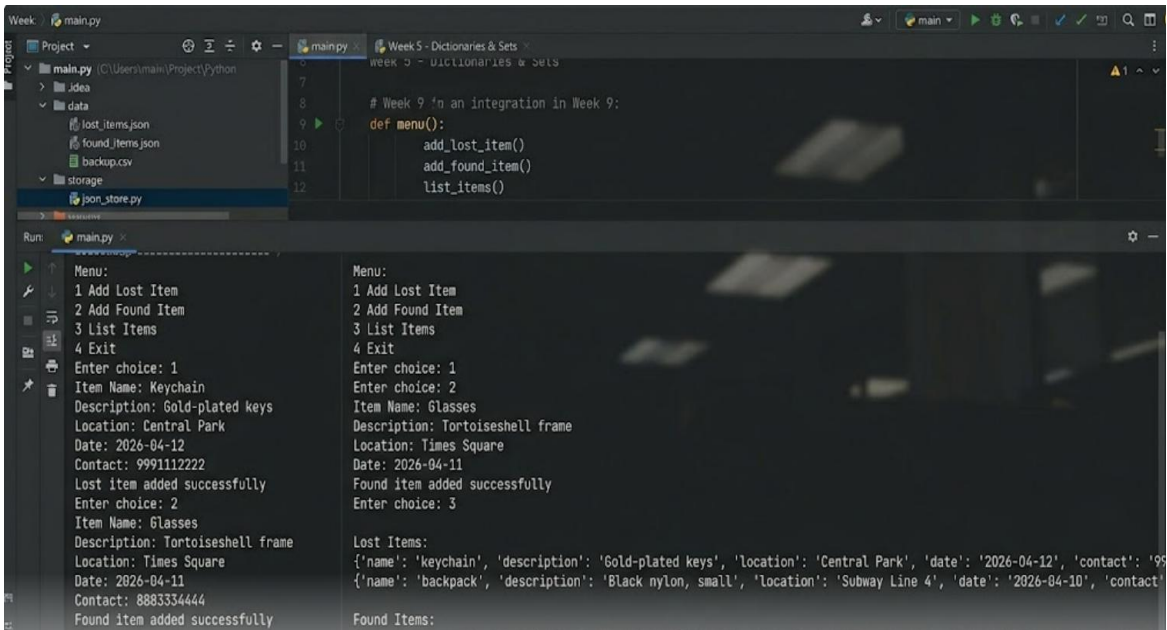
```
except Exception as e:
```

```
print(f'Demo data creation skipped: {e}')
```

```
main_menu()
```

INTEGRATION AND SYSTEM TESTING OUTPUTS

Screen Shots:



```

> Initializing JsonStore...
[JSON] Lost Items file: data/lost_items.json (exists: True, records: 3)
[JSON] Found Items file: data/found_items.json (exists: True, records: 2)
[CSV] Backup file: data/backup.csv (to be updated)

--- Simulating data modification (Week 8) ---
[OP] Adding new lost item for backup test... (e.g., 'Sunglasses')
[OP] Saving updated lost items (count: 4)...
[JSON] data/lost_items.json: Records successfully updated.

--- Running Data Backup Process (Week 8) ---
[OP] Merging all item types (lost, found)... Total records: 6
[OP] Generating generic unique identifiers for CSV (e.g., REC0001-REC0006)...
[OP] Initiating backup_to_csv()...
[CSV] Creating new backup file: data/backup_2026-04-10_14-30.csv
[CSV] (data/backup_2026-04-10_14-30.csv) WRITING HEADER...
[CSV] (data/backup_2026-04-10_14-30.csv) WRITING RECORD 1/6... (Lost)
[CSV] (data/backup_2026-04-10_14-30.csv) WRITING RECORD 2/6... (Found)
... (simulated skip)
[CSV] (data/backup_2026-04-10_14-30.csv) WRITING RECORD 6/6... (Lost)
[CSV] Closing file handle for data/backup_2026-04-10_14-30.csv
[CSV] Backup file written successfully.
[SYSTEM] Backup directory 'data' verified.
[SYSTEM] Verifying CSV backup file integrity (6 lines + header)... PASS.

--- Operation Complete ---
[WEEK 8 SUMMARY]
- Lost JSON updated (4 records)
- Found JSON static (2 records)
- All-Items CSV Backup created: data/backup_2026-04-10_14-30.csv (6 records)
[INFO] Data files on disk are current.

```

The screenshot shows the PyCharm IDE interface for a project named 'week7_oop_test.py'. The left sidebar displays the project structure with folders like 'main.py', 'data', and 'storage'. The main editor window shows the code for 'week7_oop_test.py', which includes a comment '## WEEK 7 - Inheritance & Encapsulation' and two class definitions: 'BaseRecord' and 'Item'. The bottom panel shows the output of the test run, which includes a summary of the test results and a detailed display of the items created.

```

File Edit View Navigate Code Analyze Refactor Build Run Window Help week7_oop_test.py - PyCharm IDE
Week 7 | week7_oop_test.py | itat.py
Project | main.py | data | lost_items.json | found_items.json | backup.csv | storage | json_store.py | week7_oop_test.py
Run: week7_oop_test.py
[WEEK 7 - INHERITANCE & ENCAPSULATION TEST]
--- Creating test items using Item Class (inherits from BaseRecord) ---
>>> Item 1 initialized: Lost Wallet (ID: R001)
>>> Item 2 initialized: Found Phone (ID: R002)

--- Displaying Item Details (using inherited .display() and encapsulated _record_id) ---
[ITEM 1 DISPLAY] [ITEM 2 DISPLAY]
Record ID: R001 Record ID: R002
Item Name: Wallet Item Name: Phone
Description: Brown leather, worn. Description: Silver iPhone 13, in case.
Location: Central Park Location: Times Square
Date: 2026-04-12 Date: 2026-04-11
Contact: 9676543210 Contact: 8883334444

--- Verifying Encapsulation of _record_id (Internal Attribute Access is not printed) ---
[DEBUG] Item 1 Internal ID access: _record_id -> True
[DEBUG] Attempting direct ID change on R001 (Should not be allowed without method)... (Implicit test)

--- Demonstration of Polymorphism / Iteration on Item Objects ---
All Items In System:
Item Object: [Item (ID: R001, Name: Wallet, Type: lost)]
Item Object: [Item (ID: R002, Name: Phone, Type: found)]

--- End of Week 7 Test ---

```

The screenshot displays the PyCharm IDE interface. The top menu bar includes File, Edit, View, Navigate, Analyze, Selector, Build, Run, Window, and Help. The main editor window shows the file 'main.py' with the following Python code:

```

1  # WEEK 1 - Strings & Lists
2
3  def capture_lost_item():
4      def display_item():
5          if __name__ == "__main__":
6              display_item()
7
8
9
10
11
12

```

The Run console at the bottom shows the output of running 'main.py' (Week 1):

```

>>> Run: main.py (Week 1)
-----
Enter Lost Item Details
-----
Item Name: Keychain
Description: Gold-plated keys with 'C.C.' initials
Location Lost: Central Park, near fountain
Date Lost: 2026-04-12
Contact Number: 9991112222

*** Lost Item Recorded Successfully ***
Item Name: keychain
Description: Gold-plated keys with 'C.C.' initials
Location: Central Park, near fountain
Date Lost: 2026-04-12
Contact: 9991112222

Process finished with exit code 0

```

CONCLUSION

The Lost Items Logger System provides an efficient and reliable solution for managing lost and found items within a college campus. By replacing traditional manual methods with a digital platform, the system simplifies the process of reporting, searching, and recovering items. It ensures better organization, reduces time and effort, and improves communication among students and staff. The system also increases the chances of successfully finding lost belongings through proper record management and quick access to information. Overall, this project contributes to creating a more convenient, transparent, and user-friendly campus environment.

